Practice Exercise 2 – Building an Online Travel Booking System Using Future and CompletableFuture

---

Objective

The objective of this exercise is to develop an asynchronous **Online Travel Booking System** using Java's **Future** and **CompletableFuture** APIs. Participants will learn how to execute background tasks, retrieve asynchronous results, chain dependent operations, and build modern non-blocking Java applications.

This practice exercise simulates a real-world travel booking platform where multiple booking operations occur asynchronously.

Business Scenario

You are working as a Java Developer for **SkyFly Travels**, an online flight booking company.

Every day, thousands of customers book airline tickets through the company's website and mobile application. Processing each booking involves multiple independent operations that communicate with different external systems such as airline reservation servers, payment gateways, and notification services.

Since these operations take time, the company wants the application to process them asynchronously without blocking the main application thread.

The development team has decided to use Java's **Future** and **CompletableFuture** to improve application responsiveness and simplify asynchronous programming.

Business Requirements

The application should be able to:

- Search for flight availability asynchronously.
- Allow the main application to continue performing other operations while waiting for the booking response.
- Retrieve the booking confirmation when it becomes available.
- Calculate the ticket category based on the booking amount.
- Display the final booking status.
- Notify the customer after the booking process is completed.

Project Structure

Students must create **only two Java classes**.

```
TravelBookingAssignment
└── src
    │
    ├── FlightBookingFuture.java
    ├── FlightBookingCompletableFuture.java
```

Task 1 – Implement Flight Booking Using Future

Problem Statement

The flight reservation team wants to retrieve flight booking confirmations without blocking the entire application.

Implement a Java program using **ExecutorService**, **Callable**, and **Future** that simulates an asynchronous flight booking process.

The booking operation should take approximately **4 seconds** to complete while the main thread continues performing other activities.

During the waiting period, the main thread should repeatedly display a meaningful message indicating that the customer can continue browsing other travel packages.

Once the booking completes, retrieve the booking reference using the Future object and display it on the screen.

Finally, properly shut down the ExecutorService.

Requirements

Your implementation should demonstrate the use of:

- ExecutorService
- Callable
- Future
- submit()
- Future.isDone()
- Future.get()
- shutdown()

The program should clearly show that the main thread continues executing while the booking is being processed in the background.

Task 2 – Implement Complete Booking Workflow Using CompletableFuture

Problem Statement

After the flight booking has been confirmed, several additional business operations must be performed automatically.

Instead of writing these operations sequentially, implement them using a **CompletableFuture pipeline**.

The workflow should perform the following operations asynchronously:

1. Retrieve the booking amount.
2. Determine the customer's ticket category (Economy, Business, or First Class).
3. Display the ticket category.
4. Display a booking completion message indicating that the electronic ticket has been generated successfully.

Each operation should execute automatically after the previous step completes.

Requirements

Your implementation must use:

- CompletableFuture.supplyAsync()
- thenApply()
- thenAccept()
- thenRun()
- join()

The solution should demonstrate asynchronous task chaining without manually waiting for intermediate results.

Deliverables

Students must submit the following:

- FlightBookingFuture.java
- FlightBookingCompletableFuture.java
- Program execution screenshots for both applications.

Expected Learning Outcomes

After completing this assignment, students will be able to:

- Develop asynchronous applications using ExecutorService and Future.
- Retrieve results from background tasks using Future.get().
- Monitor asynchronous task completion using Future.isDone().
- Build asynchronous processing pipelines using CompletableFuture.
- Chain dependent tasks using thenApply(), thenAccept(), and thenRun().
- Explain the limitations of Future and the advantages of CompletableFuture.
- Recommend appropriate asynchronous programming techniques for enterprise Java applications based on scalability, readability, and maintainability.